Carlos Fernando Muniz Freire - 824218484
Márcio Almeida Barrocal - 824137249
Pedro da Costa Rossaneis - 824130958

Etapa 3 - Análise Sintática e Semântica

1. Como o analisador funciona

O analisador sintático recebe os tokens do analisador léxico, confere se estão na ordem que a gramática permite e monta a árvore do programa. Depois, o analisador semântico percorre essa árvore e confere se a regra faz sentido, se as variáveis foram declaradas, se estão no bloco certo e se os tipos combinam. A regra só é aceita se passar nas três fases. Os erros mostram linha e coluna, e o analisador continua depois de cada um para o técnico ver todos de uma vez. Uma fase só roda se a anterior não teve erro.

O compilador foi passado inteiro para TypeScript 5.5 (Node.js 22), sem biblioteca de compilador; o léxico da Etapa 2 foi reescrito em lexico/lexer.ts com as mesmas expressões regulares, tokens e mensagens. Cada fase fica numa pasta. Para rodar: npm install e npm run dev.

Fase	O que verifica	Arquivo
Léxica	Os pedaços do texto são palavras válidas?	lexico/lexer.ts
Sintática	Os pedaços estão na ordem certa?	sintatico/parser.ts
Semântica	A regra faz sentido (declaração, escopo e tipos)?	semantico/semantic.ts e symbols.ts

2. Técnica usada: análise descendente

Usamos a análise descendente preditiva, LL(1), escrita à mão. Cada regra da BNF virou uma função, e o analisador decide o que fazer olhando só o próximo token. Escolhemos essa técnica porque cada comando começa com uma palavra diferente (inteiro, real ou logico, um ID, se, enquanto, leia ou escreva), então o primeiro token já diz qual regra usar, porque as mensagens de erro ficam mais claras, já que o analisador sabe o que esperava naquele ponto, e porque o código segue a ordem da BNF, o que facilita conferir com a Etapa 1.

Para funcionar na análise descendente, a gramática precisou de dois ajustes. A recursão à esquerda faria a função chamar a si mesma sem consumir token (laço infinito), então foi eliminada com $A \rightarrow A\ \alpha \mid \beta$ virando $A \rightarrow \beta\ A'$ e $A' \rightarrow \alpha\ A' \mid \varepsilon$. Já <programa> e <exp_rel> têm alternativas que começam igual, então foram fatoradas à esquerda:

Antes (Etapa 1)	Depois
<termo_ou> ::= <termo_e> | <termo_ou> "ou" <termo_e>	<termo_ou> ::= <termo_e> <termo_ou'> <termo_ou'> ::= "ou" <termo_e> <termo_ou'> | ε
<termo_e> ::= <fator_log> | <termo_e> "e" <fator_log>	<termo_e> ::= <fator_log> <termo_e'> <termo_e'> ::= "e" <fator_log> <termo_e'> | ε
<exp_arit> ::= <termo> | <exp_arit> "+" <termo> | <exp_arit> "-" <termo>	<exp_arit> ::= <termo> <exp_arit'> <exp_arit'> ::= ("+" | "-") <termo> <exp_arit'> | ε
<termo> ::= <fator> | <termo> "*" <fator> | <termo> "/" <fator>	<termo> ::= <fator> <termo'> <termo'> ::= ("*" | "/") <fator> <termo'> | ε
<programa> ::= <comando> | <comando> <programa>	<programa> ::= <comando> <programa'> <programa'> ::= <programa> | ε
<exp_rel> ::= <exp_arit> | <exp_arit> OP_REL <exp_arit>	<exp_rel> ::= <exp_arit> <exp_rel'> <exp_rel'> ::= OP_REL <exp_arit> | ε

No código, cada regra com apóstrofo virou um laço while (ou um if, em <exp_rel'>). Como o nó é montado dentro do laço, a associatividade à esquerda se mantém (8 - 3 - 2 vira (8 - 3) - 2), e a precedência continua vindo dos níveis da gramática. A linguagem aceita é a mesma da Etapa 1. Trecho de sintatico/parser.ts com a regra <exp_arit>:

```
// <exp_arit> ::= <termo> { ("+" | "-") <termo> }   (o laço é o <exp_arit'>)
private expArit(): Expr {
  let e = this.termo();
  while (this.atual.tipo === 'OP_ARIT' && (this.atual.lexema === '+' || this.atual.lexema === '-')) {
    const op = this.avancar();
    const dir = this.termo();
    e = { no: 'Binario', op: op.lexema, esq: e, dir, pos: pos(op) };
  }
  return e;
}
```

Quando encontra um erro, o analisador descarta tokens até o fim do comando e continua. No programa abaixo faltam o ";" da linha 1 e o ")" da linha 3:

```
inteiro x
x = 3 ;
se ( x > 2 {
  escreva ( x ) ;
}
```

Erro sintático (linha 1, coluna 10): esperado ';' ao final do comando, mas encontrado 'x'.

Erro sintático (linha 3, coluna 12): esperado ')' depois da condição, mas encontrado '{'.

3. Árvore sintática de dois exemplos

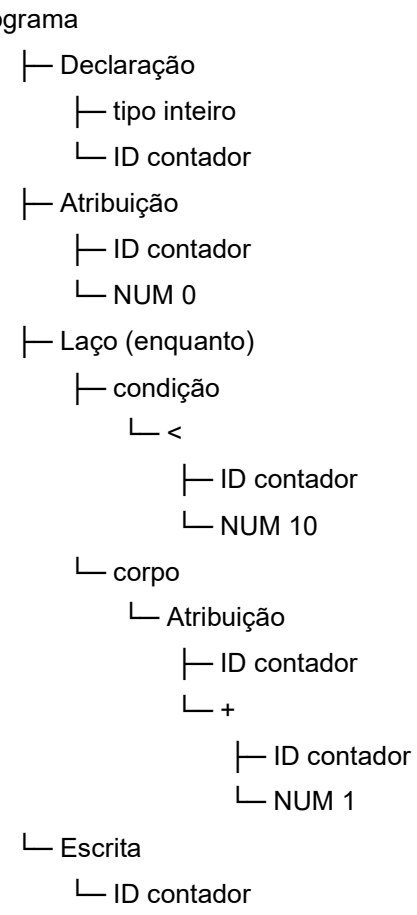
O analisador monta uma árvore sintática abstrata: o ";" e os parênteses de agrupamento não aparecem. O operador é o nó e os operandos são os filhos, então a precedência fica visível na altura dos nós. As árvores abaixo são a saída do próprio compilador.

Exemplo 1 - Contador (programa 3 da Etapa 1, com um escreva no final)

Programa:

```
inteiro contador ;
contador = 0 ;
enquanto ( contador < 10 ) {
  contador = contador + 1 ;
}
escreva ( contador ) ;
```

Árvore:



Exemplo 2 - Alerta de temperatura (programa 2 da Etapa 1)

Programa:

```
real temperatura ;
leia ( temperatura ) ;
logico alerta ;
alerta = verdadeiro ;
se ( temperatura > 35.5 e alerta == verdadeiro ) {
  escreva ( 1 ) ;
} senao {
  escreva ( 0 ) ;
}
```

Árvore:

```
Programa
    ├─ Declaração
        ├─ tipo real
        └─ ID temperatura
    ├─ Leitura
        └─ ID temperatura
    ├─ Declaração
        ├─ tipo logico
        └─ ID alerta
    ├─ Atribuição
        ├─ ID alerta
        └─ LOGICO verdadeiro
    └─ Condicional (se)
        ├─ condição
            └─ e
                ├─ >
                    ├─ ID temperatura
                    └─ NUM_REAL 35.5
                └─ ==
                    ├─ ID alerta
                    └─ LOGICO verdadeiro
        ├─ então
            └─ Escrita
                └─ NUM 1
        └─ senão
            └─ Escrita
                └─ NUM 0
```

No Exemplo 2, o e fica acima do > e do ==: as duas comparações são feitas primeiro, e só depois o e junta os resultados.

4. Tabela de símbolos e escopo

A tabela de símbolos (semantico/symbols.ts) é preenchida a cada declaração. Para cada identificador ela guarda o nome, o tipo (inteiro, real ou logico), o escopo e o nível de aninhamento, a linha e a coluna da declaração, se a variável já recebeu valor e quantas vezes foi lida.

Cada bloco { } de se, senao e enquanto abre um escopo novo, empilhado sobre o anterior, e o } fecha. Declarar coloca a variável só no escopo atual (se o nome já existe

nele, é redeclaração). Buscar procura do escopo mais interno para o mais externo, então o bloco de dentro enxerga as variáveis de fora, mas não o contrário. No exemplo abaixo, nivel só existe dentro do se:

```
real temperatura ;
leia ( temperatura ) ;
se ( temperatura > 30.0 ) {
  inteiro nivel ;
  nivel = 2 ;
  escreva ( nivel ) ;
}
```

Nome	Tipo	Escopo	Declarada em	Recebe valor	Usos
temperatura	real	global	1:6	sim	1
nivel	inteiro	bloco 1 (se, linha 3)	4:11	sim	1

5. Verificações semânticas

Um inteiro pode ir para uma variável real, mas um real não pode ir para uma inteira. Os operadores aritméticos, < e > pedem números; e, ou, nao e as condições de se e enquanto pedem logico. Depois de um erro, a expressão recebe o tipo "erro", para um engano não gerar várias mensagens repetidas.

Verificação 1 - Variável não declarada

```
inteiro idade ;
idade = 30 ;
total = idade * 2 ;
```

Mensagem: Erro semântico (linha 3, coluna 1): variável 'total' não foi declarada.

Verificação 2 - Incompatibilidade de tipos

```
inteiro idade ;
logico alerta ;
idade = 30 ;
alerta = idade + 1 ;
```

Mensagem: Erro semântico (linha 4, coluna 10): tipo incompatível: 'alerta' é logico, mas a expressão é inteiro.

Verificação 3 - Uso fora do escopo

```
se ( verdadeiro ) {
  inteiro n ;
  n = 1 ;
}
escreva ( n ) ;
```

Mensagem: Erro semântico (linha 5, coluna 11): variável 'n' não foi declarada.

Também são verificados: real atribuído a inteiro (perda de precisão), redeclaração no mesmo escopo, condição de se ou enquanto que não é logico, operador com tipo errado, == entre tipos diferentes e divisão por zero constante. Variável lida antes de receber valor gera só um aviso.

6. Por que essas verificações não podem ser feitas pela gramática

A BNF da Etapa 1 é livre de contexto (Tipo 2): cada regra é aplicada sem olhar o que existe em volta, e para o analisador sintático ID é só um token. Verificar se uma variável foi declarada exige ligar duas ocorrências do mesmo nome, separadas por qualquer quantidade de comandos, que é o mesmo problema da linguagem $\{ w\ c\ w \mid w \in \{a, b\}^* \}$, que não é livre de contexto (lema do bombeamento). O tipo e o escopo dependem dessa mesma informação distante. Essas restrições pertencem às gramáticas sensíveis ao contexto (Tipo 1); na prática, usa-se uma gramática de atributos (Knuth, 1968), em que a árvore recebe atributos, como o tipo, conferidos com a ajuda da tabela de símbolos. É isso que o semantic.ts faz.

7. Sintaxe e semântica são responsabilidades diferentes

As duas fases ficam separadas de propósito. x = 3 + ; tem erro de forma, e x = 3 + verdadeiro ; tem a forma certa mas não faz sentido: são problemas diferentes, com mensagens diferentes. Além disso, se há erro de sintaxe a árvore fica incompleta e a semântica nem roda, o que evita erros falsos.

	Analisador sintático	**Analisador semântico**
Recebe	os tokens	a árvore pronta
Entrega	a árvore (ou erros de forma)	a tabela de símbolos e os erros de significado
Conhece	só a gramática	declarações, escopos e tipos